2026-05-16 - Introduction to Zephyr RTOS

Goal

Learn what Zephyr is, why it exists, and how it differs from other embedded RTOS options.

What you will learn

- What Zephyr RTOS is and its core use cases
- How Zephyr supports multiple architectures and boards
- The difference between Zephyr application code and Zephyr kernel/SDK
- The role of West, CMake, Kconfig, and Device Tree in Zephyr

Overview

Zephyr is a small, scalable real-time operating system for embedded devices. It is designed for constrained hardware and supports a wide range of microcontrollers, sensors, networking stacks, and boards.

Key concepts

- **Kernel**: task scheduling, threads, synchronization, and timers
- **Subsystems**: networking, file systems, Bluetooth, sensor frameworks
- **Build system**: Zephyr uses CMake and West to manage projects and dependencies
- **Configuration**: Kconfig controls compile-time features and board-specific settings
- **Device Tree**: describes hardware peripherals and pin mappings

Why this matters

Understanding these concepts helps you read Zephyr examples, choose the right board and SDK, and avoid common mistakes when starting a new project.

Practice tasks

1. Inspect the Zephyr repository or samples to identify core directories: `zephyr/kernel`, `zephyr/subsys`, `zephyr/samples`.
2. Compare Zephyr to a bare-metal project: note what Zephyr provides out of the box.
3. Make a list of one board and one feature you want to explore next.

Next topic

Tomorrow, we will explore Zephyr project structure and the build system in depth.